Okno ze zdjęcia – wersja 2D (PixiJS)

Plan technologii i kolejnych kroków · listopad 2026

1. Pomysł w jednym zdaniu

Użytkownik wrzuca dowolne zdjęcie, a aplikacja – dzięki AI, które rozpoznaje głębię i niebo – zamienia je w „okno hotelowe” z paralaksą, porą dnia i pogodą. Wersja 2D: obraz zostaje płaski, a piksele są przesuwane zależnie od głębi (filtr w PixiJS).

2. Jak działa aplikacja (oczami użytkownika)

1. Użytkownik klika **„Zrób okno z własnego zdjęcia”** i wrzuca zwykłe zdjęcie (JPG/PNG).
2. Aplikacja pokazuje „Analizuję zdjęcie…” z paskiem postępu. W tym czasie w przeglądarce działają modele AI.
3. **AI nr 1 – głębia** (Depth Anything V2): dla każdego piksela ocenia, jak daleko jest od aparatu.
4. **AI nr 2 – niebo** (segmentacja SegFormer): zaznacza, gdzie na zdjęciu jest niebo.
5. Zdjęcie zamienia się w okno: ruch myszki / przechył telefonu / **ruch głowy** (AI nr 3 – MediaPipe, wykrywanie twarzy z kamerki) przesuwa widok z efektem głębi.
6. Niebo ze zdjęcia jest podmienione na animowane niebo z projektu: działa suwak pory dnia (zachód, noc, gwiazdy) i pogoda.
7. Opcjonalnie **AI nr 4 – Claude** opisuje zdjęcie i dobiera dźwięki oraz klimat (jedyna płatna część).

3. Co dokładnie robi AI

AI	Wejście	Wyjście	Do czego	Koszt
Depth Anything V2 (small)	zdjęcie	mapa głębi (szara: jasne = blisko)	efekt głębi przy ruchu	0 zł, w przeglądarce
SegFormer (ADE20K, klasa „sky”)	zdjęcie	maska nieba	podmiana nieba → dzień/noc, pogoda	0 zł, w przeglądarce
MediaPipe Face Landmarker	obraz z kamerki	pozycja głowy	sterowanie widokiem głową	0 zł, w przeglądarce
Claude API (opcjonalnie)	zdjęcie	opis + dobór dźwięków/klimatu	dopasowanie atmosfery	płatne, grosze/zapytanie, wymaga backendu

Cała reszta – paralaksa, wygładzanie ruchu, niebo, dzień/noc, pogoda, UI – to **wasz własny kod**.

4. Technologie

Warstwa	Technologia	Rola
Render	PixiJS 8 (już w projekcie)	Sprite zdjęcia, DisplacementFilter (przesunięcie pikseli wg głębi), ColorMatrixFilter (dzień/noc), ParticleContainer (deszcz/śnieg)
AI w przeglądarce	@huggingface/transformers (transformers.js)	Uruchamia Depth Anything i SegFormer (ONNX, WebGPU/WASM)
Kamera	@mediapipe/tasks-vision	Face Landmarker – pozycja głowy (dozwolone w CLAUDE.md, Etap 9)
Wątek w tle	Web Worker (wbudowany w przeglądarkę)	Modele AI liczą się poza wątkiem UI

Warstwa	Technologia	Rola
Język / build / testy	TypeScript strict, Vite, Vitest	Bez zmian względem roadmapy
Opcjonalnie	Claude API + Cloudflare Worker	Opis zdjęcia i dobór klimatu; backend chroni klucz API

Nowa zależność @huggingface/transformers – wg zasad CLAUDE.md trzeba ją najpierw zatwierdzić i dopisać do listy dozwolonych.

5. Jak to działa technicznie

Paralaksa per piksel

DisplacementFilter czyta mapę głębi i przesuwa każdy piksel o: **(głębia − 0.5) × pozycja widza × siła**. Bliskie punkty idą w jedną stronę, dalekie w drugą, środek stoi. To ten sam wzór co w ParallaxStage, tylko liczony dla każdego piksela, a nie dla warstwy.

```
const filter = new DisplacementFilter({ sprite: depthSprite, scale: 0 });
photo.filters = [filter];
// every frame, viewer in -1..1 from InputManager
filter.scale.x = viewer.x * strength;
filter.scale.y = viewer.y * strength;
```

Podmiana nieba

Maska nieba służy jako maska sprite'a: tam, gdzie było niebo, widać wasze animowane niebo (gradient, słońce, księżyc, gwiazdy, chmury). Resztę zdjęcia przyciemnia/barwi ColorMatrixFilter zależnie od godziny.

Pogoda „w głębi”

Mapę głębi progujecie na 2 maski: *blisko* i *daleko*. Deszcz/śnieg rysujecie w dwóch warstwach: dalsza zamaskowana tak, by była zasłonięta przez bliskie obiekty, bliższa na wierzchu. Daje to wrażenie, że pada między obiektami.

Proponowane pliki

```
src/ai/depthWorker.ts      # Web Worker: load model, return depth map
src/ai/depth.ts            # API: estimateDepth(image) -> ImageData
src/ai/sky.ts              # API: segmentSky(image) -> mask
src/ai/depthUtils.ts       # pure functions: normalize, blur, thresholds (Vitest)
src/engine/DepthPhoto.ts   # PixiJS: sprite + DisplacementFilter
src/effects/SkyReplace.ts  # PixiJS: mask + animated sky
src/ui/PhotoUpload.ts      # upload button, progress bar, errors
```

6. Co robić dalej – kroki

Krok 0 – Decyzja (1 spotkanie)

- Ustalcie z Marcelem (i prowadzącym), że ta funkcja wchodzi do projektu.
- Zatwierdźcie zależność @huggingface/transformers i dopiszcie ją do CLAUDE.md.

» **Gotowe, gdy:** decyzja zapisana w CLAUDE.md, nowy etap dopisany do roadmapy.

Krok 1 – Prototyp głębi (1–2 dni)

- Strona testowa: wrzuć zdjęcie → pokaż mapę głębi obok.
- Sprawdźcie na 5–10 waszych zdjęciach, jak model sobie radzi.

» **Gotowe, gdy:** widać sensowną mapę głębi; wiecie, jakie zdjęcia działają dobrze.

Krok 2 – Paralaksa w PixiJS (1–2 dni)

- DepthPhoto.ts z DisplacementFilter, podpięty pod InputManager (myszka/żyroskop).
- Dobór siły efektu, lekkie rozmycie mapy głębi (mniej artefaktów).

» **Gotowe, gdy:** ruch myszką daje przekonujący efekt głębi bez mocnego „gumowania”.

Krok 3 – Web Worker + UI (1–2 dni)

- Przeniesienie model do Web Workera, pasek postępu pobierania i analizy.
- Obsługa błędów: za duże zdjęcie, brak WebGPU (fallback WASM), przerwane pobieranie.

» **Gotowe, gdy:** UI się nie zacina, błędy nie wywalają aplikacji.

Krok 4 – Podmiana nieba + dzień/noc (2–3 dni)

- sky.ts (SegFormer), maska nieba w PixiJS, podpięcie TimeOfDay z Etapu 4.

» **Gotowe, gdy:** suwak pory dnia zmienia niebo i koloryt dowolnego zdjęcia.

Krok 5 – Pogoda w głębi (2 dni)

- Progowanie mapy głębi, deszcz/śnieg w dwóch warstwach (Etap 5).

» **Gotowe, gdy:** deszcz pada „między" obiektami, nie tylko na wierzchu.

Krok 6 – Kamera (głowa) (2–3 dni)

- CameraInput.ts z MediaPipe (Etap 9), mocne wygładzanie, fallback do myszki.

» **Gotowe, gdy:** ruch głową przed laptopem przesuwa widok.

Krok 7 – Opcjonalnie Claude (2 dni)

- Cloudflare Worker z kluczem API, endpoint „opisz zdjęcie → JSON klimatu", walidacja JSON.

» **Gotowe, gdy:** zdjęcie plaży automatycznie dostaje fale i mewy.

Krok 8 – Testy i prezentacja (2 dni)

- Vitest dla depthUtils, testy na iPhonie/Androidzie, demo z waszym zdjęciem.

» **Gotowe, gdy:** build, lint, test przechodzą; działa na telefonie.

7. Ryzyka i ograniczenia

- **Rozmiar modeli:** pierwsze pobranie to kilkadziesiąt MB (potem cache przeglądarki). Ładować dopiero po kliknięciu „wrzuć zdjęcie".
- **Czas analizy:** komputer ok. 1–3 s, telefon ok. 5–15 s. Robione raz na zdjęcie, w Web Workerze (UI się nie zacina).
- **Trudne zdjęcia:** szkło, woda, mgła, drobne gałęzie, włosy – mapa głębi bywa błędna. Płaskie zdjęcia (ściana) dają słaby efekt.
- **Klucz Claude API** nigdy w kodzie frontendu (GitHub Pages jest publiczne) – tylko przez mały backend (np. Cloudflare Worker).
- **Nazwy modeli i API bibliotek** w tym dokumencie należy zweryfikować w aktualnej dokumentacji przed implementacją.
- **Specyficzne dla 2D:** przy większym ruchu widza widać rozciąganie na krawędziach obiektów („guma"). Rozwiązanie: mała siła efektu (ok. 10–20 px) i rozmyta mapa głębi.
- **Brak prawdziwej perspektywy:** obiekty się „ślizgają", nie obracają.

8. Podsumowanie

	Wersja 2D (PixiJS)
Trudność	niska–średnia
Jakość efektu	dobra przy małym ruchu
Zgodność z CLAUDE.md	tak (po zatwierdzeniu transformers.js)
Koszt	0 zł (bez opcjonalnego Claude)
Szacowany czas	ok. 2–3 tygodnie pracy w dwie osoby